ECOLE SUPERIEURE MULTINATIONALE DES TELECOMMUNICATIONS



PROJET BASE DE DONNEES NOSQL :

Etude et mise en œuvre du ReplicaSet avec MongoDB

PRODUIT PAR :

BOUGOUM Cheick

HOUNKANRIN Fahouk

SISSAO Grace

FORMATEUR :

M. PREIRA

INGC-INDIA-ESMT

SOMMAIRE

INTRODUCTION 4

1. Contexte 4

2. Problématique 5

3. Objectifs 5

I. LES CONCEPTS 6

1. Architecture d'un Replica Set MongoDB 6

1.1. Vue d'ensemble 6

1.2. Le primary/primaire 6

1.3. Le secondary/ secondaire 6

1.4. L'arbiter/ arbitre 6

2. Mécanisme d'élection et tolérance aux pannes 7

2.1. Les heartbeats 7

2.2. Déclenchement d'une élection 7

2.3. Tolérance aux pannes 8

2.4. Split-Brain 9

3. L'oplog (opération log) 9

- **Structure d'une entrée d'oplog :** 10

4. Read / Write Concerns 10

- **Write Concern** 10
- **Read Concern** 11
- **Read Preference** 11

II. CONFIGURATION ET DEPLOIEMENT 12

1. Environnement utilisé 13

2. Installation de MongoDB 13

- **Sur Windows** 13

3. Création des répertoires de données 13

4. Lancement de chaque nœud 17

Terminal 1 – Noeud 1 (futur Primary) 17

Terminal 2 - Noeud 2 (Secondary) 17

Terminal 3 - Noeud 3 (Secondary) 17

5. Initialisation du réplicaset 18

6. Ajout des nœuds au ReplicaSet 18

7. Vérification des statuts du ReplicaSet 19

III. Tests et Résultats 21

1. **Test 1 — Réplication des données** 21
Description du test 21
Procédure 21
Résultats et Observations 21
2. **Test 2 — Failover (Panne du Primary)** 22
Description du test 22
Procédure 22
Résultats et Observations 23
3. **Test 3 — Réintégration d'un nœud** 23
Description du test 23
Procédure 23
Résultats et Observations 24
4. **Test 4 — Lecture sur les nœuds Secondaires** 24
Description du test 24
Procédure 24
Résultats et Observations 25
IV. LIMITES ET PERSPECTIVES 25
1. **Limites du ReplicaSet seul** 25
- Limites en matière d'écritures 25
- Limites en matière de volume de données 26
- Autres limites 26
2. **Combinaison possible avec le Sharding** 26
Architecture d'un Sharded Cluster MongoDB 26
Quand passer au Sharded Cluster ? 27
CONCLUSION 28

INTRODUCTION

1. Contexte

Nous vivons aujourd'hui dans une ère où les données constituent le cœur des systèmes d'information modernes. Des applications bancaires aux plateformes de commerce électronique, en passant par les systèmes de santé et les réseaux sociaux, la demande en disponibilité continue et en fiabilité des données n'a jamais été aussi élevée.

Face à cette réalité, MongoDB s'est imposé comme l'un des systèmes de gestion de bases de données NoSQL les plus adoptés au monde. Selon le classement DB-Engines de 2024, MongoDB occupe la première place des bases de données orientées documents et la cinquième place toutes catégories confondues. Sa flexibilité, sa scalabilité et sa capacité à gérer de grands volumes de données non structurées en font un choix privilégié pour les architectures modernes.

Cependant, toute infrastructure reposant sur un serveur unique présente des risques critiques : une panne matérielle, une mise à jour défaillante ou une surcharge du système peut rendre l'ensemble de l'application inaccessible et entraîner des pertes de données irréversibles. Pour répondre à ces risques, MongoDB propose un mécanisme natif de réplication appelé **ReplicaSet**.

2. Problématique

Dans un environnement de production, s'appuyer sur une instance MongoDB unique soulève plusieurs problèmes fondamentaux : **la disponibilité, la durabilité des données**, l**a continuité de service**, l**es performances en lecture**.

Ces constats nous amènent à poser la problématique centrale de ce projet :

Comment garantir la haute disponibilité, la tolérance aux pannes et la continuité de service d'une base de données MongoDB dans un environnement de production, tout en assurant la cohérence et l'intégrité des données ?

3. Objectifs

Objectif principal : Comprendre, étudier et mettre en œuvre un replicaset avec MongoDB

Ce projet a pour objectifs de :

- Comprendre les fondements théoriques de la réplication dans MongoDB.
- Etudier l'architecture et les mécanismes du ReplicaSet MongoDB.
- Mettre en œuvre un ReplicaSet fonctionnel avec trois nœuds sur un environnement local.
- Tester et valider les comportements clés : réplication automatique, élection en cas de panne, et réintégration d'un nœud.
- Analyser les résultats obtenus et tirer des conclusions sur les avantages et limites de cette approche.

I. LES CONCEPTS

1. Définitions

1.1. Qu'est ce que la réplication ?

La réplication est le processus de synchronisation des données sur plusieurs serveurs. Elle consiste à maintenir plusieurs copies identiques d'un même jeu de données sur des nœuds différents, en temps quasi-réel. Elle garantit la disponibilité continue des données en cas de panne et améliore les performances en lecture en distribuant les requêtes sur plusieurs nœuds.

1.2. La réplication VS la sauvegarde

Ces deux concepts sont souvent confondus, mais ils répondent à des besoins distincts :

Critère	Réplication	Sauvegarde
Objectif	Haute disponibilité	Récupération après sinistre
Délai	Temps réel (quasi-immédiat)	Ponctuelle (horaire, quotidienne…)
Protection	Panne matérielle, perte de nœud	Corruption, suppression accidentelle
Limite	Données corrompues aussi répliquées	Permet de revenir à un état antérieur
Automatisation	Automatique et continue	Planifiée ou manuelle

1.3. La réplication VS le sharding

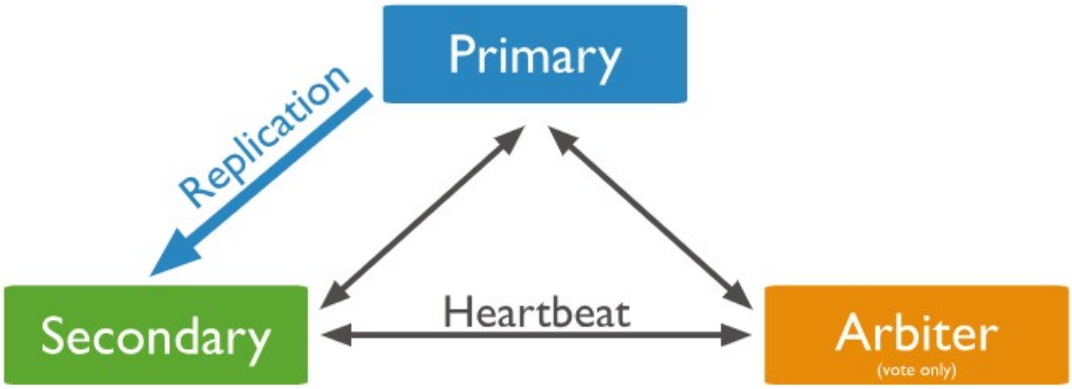
Critère	Replica Set	Sharding
Objectif principal	Haute disponibilité et redondance	Scalabilité horizontale
Données	Chaque nœud contient toutes les données	Chaque shard contient une partition
Cas d'usage	Tolérance aux pannes	Très grands volumes de données
Complexité	Faible à modérée	Élevée
Lecture/Écriture	Écritures sur le Primary uniquement	Distribuées sur les shards

> 💡 *En pratique, un cluster shard MongoDB est composé de plusieurs replica sets : chaque shard est lui-même un replica set.*

2. Architecture d'un Replica Set MongoDB

2.1. Vue d’ensemble

Un ReplicaSet est un groupe de processus **mongod** qui maintiennent le même jeu de données. Il est composé de trois types de nœuds :



2.2. Le primary/primaire

C'est le seul nœud qui accepte les opérations d'écriture. Il enregistre toutes les modifications dans son oplog, que les secondaires vont lire pour se synchroniser. Il n'y a toujours qu'un seul Primary à la fois dans un ReplicaSet.

2.3. Le secondary/ secondaire

Les secondaires répliquent l'oplog du Primary et appliquent les opérations en asynchrone. Ils peuvent servir des lectures (si configuré) et sont éligibles pour devenir Primary en cas de panne. Un ReplicaSet peut avoir jusqu'à 50 membres, dont 7 votants maximum.

2.4. L’arbiter/ arbitre

L'arbiter est un nœud léger qui ne stocke aucune donnée. Son seul rôle est de participer aux votes d'élection pour éviter les situations d'égalité (split-brain).

> ⚠️ *L'arbiter n'est pas recommandé en production pour des données sensibles car il n'offre aucune redondance des données.*

3. Mécanisme d’élection et tolérance aux pannes

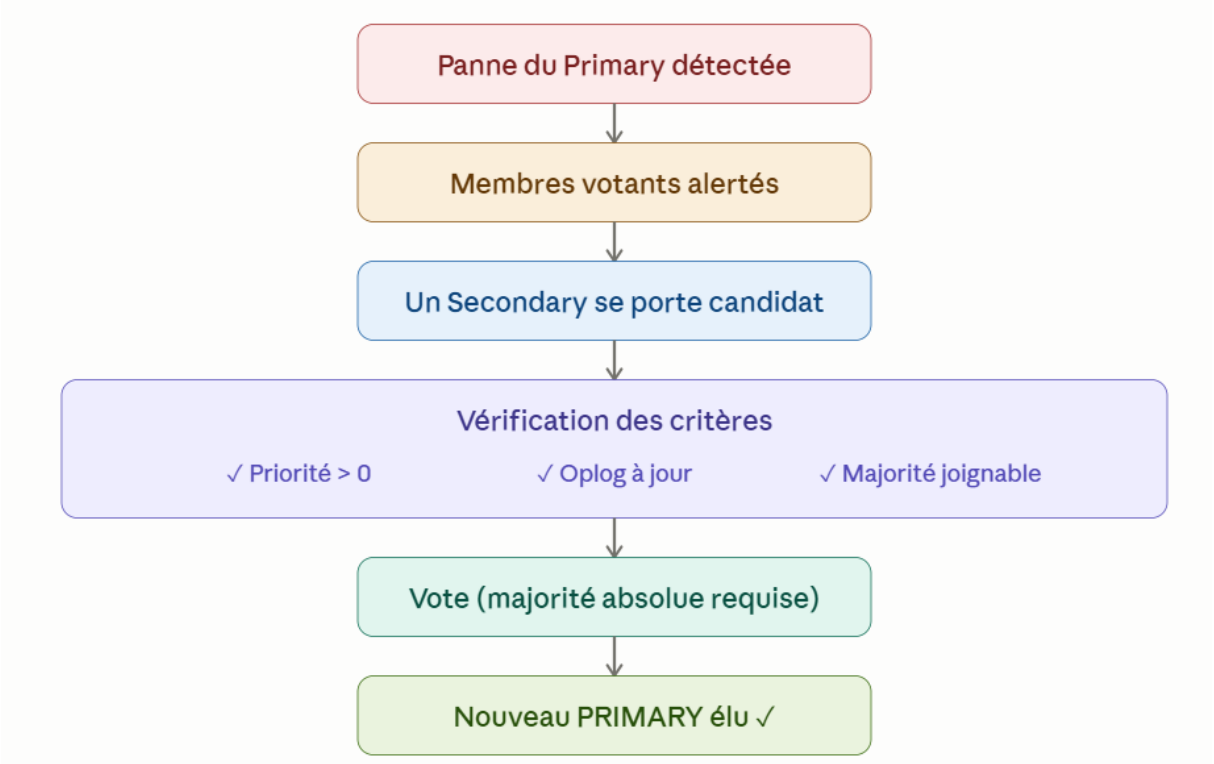
3.1. Les heartbeats

Chaque membre envoie des heartbeats (pings) à tous les autres membres toutes les 2 secondes. Si un membre ne répond pas dans un délai de 10 secondes (electionTimeoutMillis), il est considéré comme inaccessible.

3.2. Déclenchement d’une élection

Une élection est déclenchée quand :

- Le Primary ne répond plus aux heartbeats
- Un Secondary détecte qu'il a un oplog plus récent que le Primary
- Au démarrage du ReplicaSet



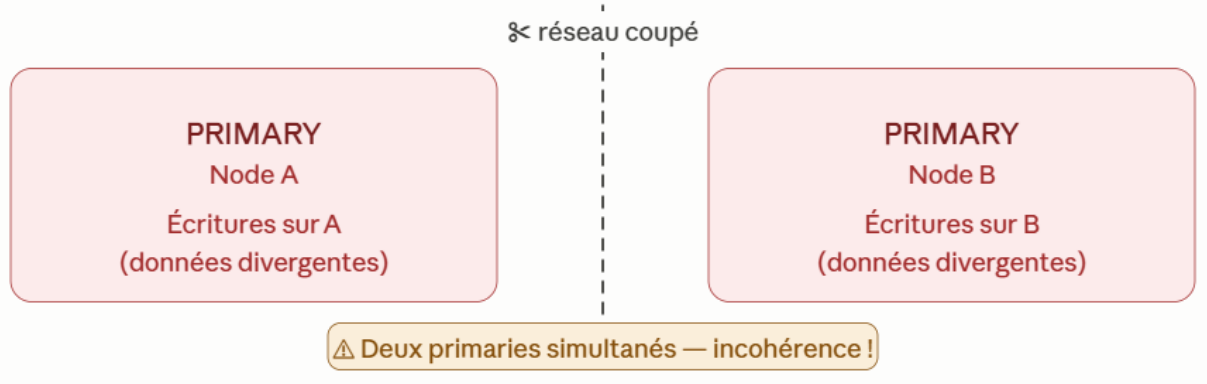
3.3. Tolérance aux pannes

Taille du Replica Set	Majorité requise	Pannes tolérées
3 membres	2	1
5 membres	3	2
7 membres	4	3

⚠️ *Si la majorité n'est plus atteignable, le Primary se rétrograde en Secondary automatiquement pour éviter des écritures en mode split-brain.*

3.4. Split-Brain

Le split-brain est une situation où, suite à une partition réseau, deux sous-groupes de nœuds se croient mutuellement morts et tentent chacun de fonctionner de manière autonome avec deux primaries/primaires simultanés.



MongoDB évite le split-brain grâce au quorum : un Primary ne peut exister que s'il est reconnu par une majorité absolue des membres votants. Le groupe minoritaire ne peut pas élire un nouveau Primary et reste en lecture seule.

Mécanisme	Rôle contre le split-brain
Règle de majorité	Empêche l'élection d'un second Primary
Heartbeats (2s / 10s)	Détecte rapidement la panne
w: majority	Empêche les écritures non répliquées d'être confirmées
Rollback automatique	Corrige les divergences résiduelles à la reconnexion

4. L'oplog (opération log)

- **Définition** : L'oplog est une collection spéciale cappée (local.oplog.rs) présente sur chaque membre. C'est le journal de toutes les opérations d'écriture effectuées sur le Primary.
- **Structure d'une entrée d'oplog :**

Champ	Valeur exemple	Description
"ts"	`Timestamp(1687000000, 1)`	Horodatage de l'opération (date + compteur ordinal)
"t"	`42`	Numéro du terme d'élection (mandat du Primary actuel)
"op"	`"i"`	Type d'opération i = insert · u = update · d = delete
"ns"	`"mydb.users"`	Namespace cible base.collection concernée
"o"	`{ "_id": 1, "name": "Ada" }`	Document concerné par l'opération

- **Fonctionnement :** Le primary écrit chaque opération dans l'oplog
- Les secondaires tirent (pull) les entrées de l'oplog du primary de façon continue
- Les opérations sont idempotentes : elles peuvent être rejouées plusieurs fois sans effet de bord

- L'oplog a une taille limitée (~5% du disque disponible) : une fois plein, les vieilles entrées sont écrasées

⚠️ *Si un Secondary est trop longtemps hors ligne et que ses dernières opérations ont été écrasées dans l'oplog, il tombe en état RECOVERING et doit être resynchronisé manuellement (initial sync).*

5. Read / Write Concerns

➢ Write Concern

Le write concern définit le niveau d'accusé de réception exigé avant que MongoDB ne confirme une écriture à l'application.

Valeur	Signification	Durabilité
w: 0	Pas d'accusé de réception (fire & forget)	Très faible
w: 1	Confirmation du Primary seulement (défaut)	Faible
w: majority	Confirmation de la majorité des votants	Fort
w: n	Confirmation de n membres	Variable
j: true	Écriture persistée sur le journal disque	Durabilité physique

➢ Read Concern

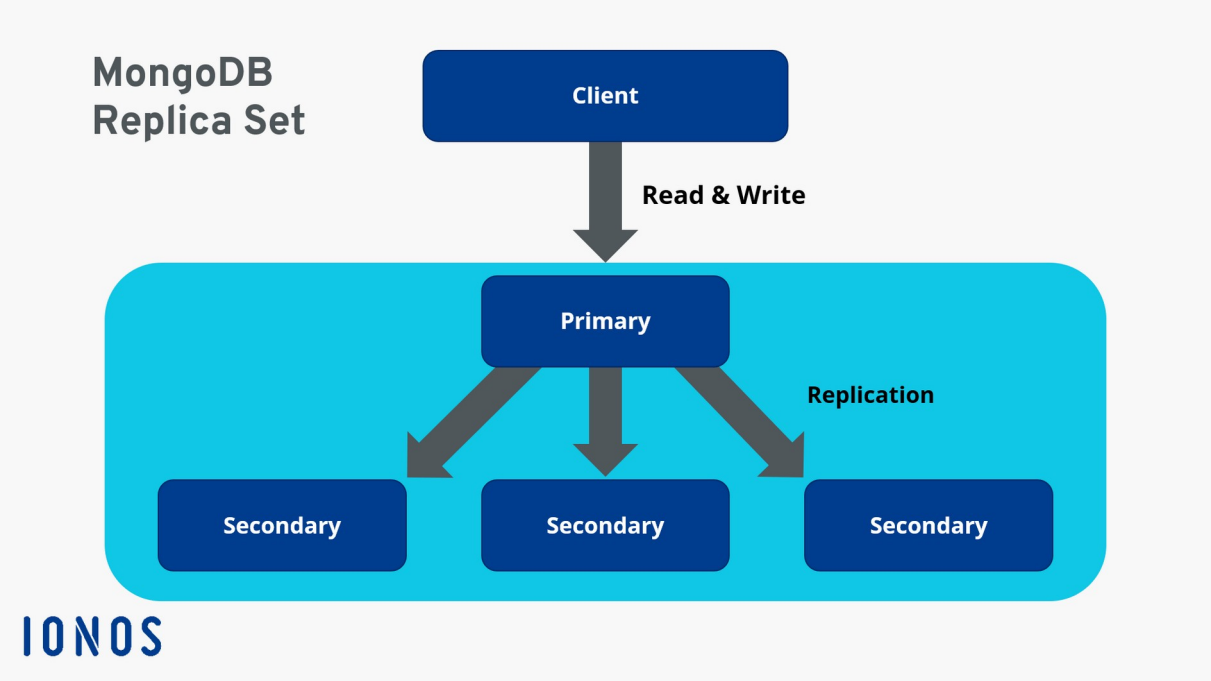
Le read concern définit la fraîcheur et la cohérence des données retournées lors d'une lecture.

Niveau	Description	Cas d'usage
local	Données locales du nœud (défaut)	Lectures non critiques
available	Comme local sans garantie de cohérence	Sharding uniquement
majority	Données confirmées par la majorité	Lectures critiques
linearizable	Lecture la plus récente et confirmée	Données financières
snapshot	Vue cohérente à un instant T	Transactions ACID

➢ Read Preference

Ce paramètre définit sur quel membre la lecture est routée :

Mode	Description
primary	Toujours sur le Primary (défaut)
primaryPreferred	Primary si disponible, sinon Secondary
secondary	Toujours sur un Secondary
secondaryPreferred	Secondary si disponible, sinon Primary
nearest	Le nœud avec la latence réseau la plus faible



II. CONFIGURATION ET DEPLOIEMENT

1. Environnement utilisé

Composant	Version / Valeur	Remarque
Système d'exploitation	Windows 11	Environnement local
Version MongoDB	MongoDB Community 8.2.6	Version utilisée pour les tests
MongoDB Shell	mongosh 2.8.2	Interface en ligne de commande
MongoDB Compass	Version 1.42	Interface graphique (optionnel)
Architecture	1 machine, 3 membres + 1 instance arbitre	Simulation via ports distincts

Architecture déployée sur une seule machine : trois membres de données et une instance arbitre, sur des ports différents :

- Noeud 1 (Primary) : localhost:27017
- Noeud 2 (Secondary) : localhost:27018
- Noeud 3 (Secondary) : localhost:27019
- Noeud 4 (Arbiter, service installé mais absent de la configuration d'initialisation ci-dessous) : localhost:27020

2. Installation de MongoDB

➢ **Sur Windows**

- Télécharger l'installeur MSI depuis le site officiel

https://www.mongodb.com/try/download/community et suivre l'assistant d'installation.

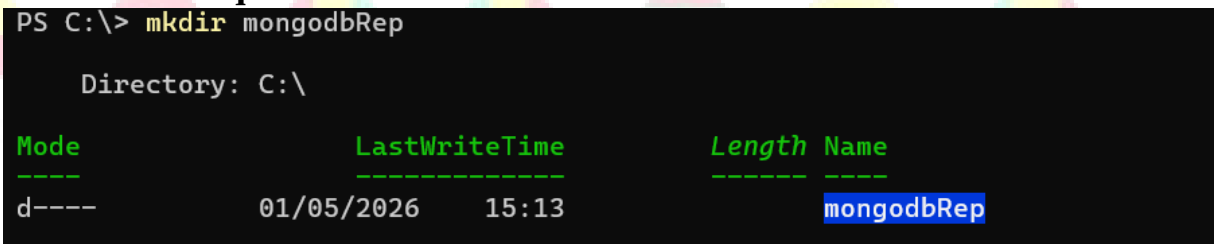
- Ajouter le répertoire **bin** de MongoDB au **PATH** système.

3. Création des répertoires de données

Chaque instance MongoDB nécessite son propre répertoire de stockage :

Sur Windows (PowerShell) :

➢ **Création du répertoire principal qui contiendra tous les fichiers du TP dans le disque local C**



➢ Création des différents nœuds et de l'arbitre

```
PS C:\> cd /mongodbRep
PS C:\mongodbRep> mkdir -p ./data/rs1, ./data/rs2, ./data/rs3, ./data/arb


    Directory: C:\mongodbRep\data

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d----          01/05/2026    15:13                rs1
d----          01/05/2026    15:13                rs2
d----          01/05/2026    15:13                rs3
d----          01/05/2026    15:13                arb
```

➢ Création des répertoires logs et config dans le répertoire principal

```
PS C:\mongodbRep> mkdir -p ./logs/mongodb


    Directory: C:\mongodbRep\logs

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d----          01/05/2026    15:13                mongodb
```

```
PS C:\mongodbRep> mkdir -p ./mongodb/config


    Directory: C:\mongodbRep\mongodb

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d----          01/05/2026    15:14                config
```

➢ Contenu du nœud rs1.cfig

rs1.cfg | rs2.cfg | rs3.cfg | arbiter.cfg

C: > mongodbRep > mongodb > config > rs1.cfg

```
storage:
  dbPath: C:\mongodbRep\data\rs1

systemLog:
  destination: file
  path: C:\mongodbRep\logs\mongodb\rs1.log
  logAppend: true

net:
  port: 27017
  bindIp: 127.0.0.1

replication:
  replSetName: "rs0"

#security:
#  keyFile: C:\mongodb\keyfile.txt
```

➢ **Contenu du nœud rs2.cfg**

rs1.cfg | rs2.cfg | rs3.cfg | arbiter.cfg

C: > mongodbRep > mongodb > config > rs2.cfg

```
storage:
  dbPath: C:\mongodbRep\data\rs2

systemLog:
  destination: file
  path: C:\mongodbRep\logs\mongodb\rs2.log
  logAppend: true

net:
  port: 27018
  bindIp: 127.0.0.1

replication:
  replSetName: "rs0"

#security:
#  keyFile: C:\mongodb\keyfile.txt
```

➢ **Contenu du nœud rs3.cfg**

rs1.cfg | rs2.cfg | rs3.cfg | arbiter.cfg

C: > mongodbRep > mongodb > config > rs3.cfg

```
storage:
  dbPath: C:\mongodbRep\data\rs3

systemLog:
  destination: file
  path: C:\mongodbRep\logs\mongodb\rs3.log
  logAppend: true

net:
  port: 27019
  bindIp: 127.0.0.1

replication:
  replSetName: "rs0"

#security:
#  keyFile: C:\mongodb\keyfile.txt
```

➢ **Contenu du nœud arbitre.cfg**

rs1.cfg | rs2.cfg | rs3.cfg | arbiter.cfg

C: > mongodbRep > mongodb > config > arbiter.cfg

```
storage:
  dbPath: C:\mongodbRep\data\arb

systemLog:
  destination: file
  path: C:\mongodbRep\logs\mongodb\arbiter.log
  logAppend: true

net:
  port: 27020
  bindIp: 127.0.0.1

replication:
  replSetName: "rs0"

#security:
#  keyFile: C:\mongodb\keyfile.txt
```

NB : Cette partie est nécessaire seulement lorsqu'on utilise les services pour chaque nœud sinon elle est facultative

4. Lancement de chaque nœud

Méthode 1 : Démarrer les instances en ligne de commande

Ouvrir trois terminaux distincts et lancer une instance dans chacun :

Terminal 1 – Nœud 1 (futur Primary)

```
PS C:\> cd ./mongodbRep
PS C:\mongodbRep> mongod --replSet rs0 --port 27017 --dbpath C:\mongodbRep\data\rs1 --bind_ip localhost
```

Terminal 2 - Nœud 2 (Secondary)

```
PS C:\> cd ./mongodbRep
PS C:\mongodbRep> mongod --replSet rs0 --port 27018 --dbpath C:\mongodbRep\data\rs1 --bind_ip localhost
```

Terminal 3 - Nœud 3 (Secondary)

```
PS C:\> cd ./mongodbRep
PS C:\mongodbRep> mongod --replSet rs0 --port 27019 --dbpath C:\mongodbRep\data\rs1 --bind_ip localhost
```

Note : dans ces trois captures, l'option --dbpath désigne le même dossier (data\rs1). Pour lancer trois instances en parallèle, chaque nœud doit avoir son propre dossier : data\rs1, data\rs2 et data\rs3, comme dans les fichiers de configuration de la méthode 2.

Paramètre --replSet

Le paramètre --replSet rs0 indique à chaque instance MongoDB qu'elle fait partie d'un ReplicaSet nomme 'rs0'. Toutes les instances d'un même groupe doivent utiliser le même nom.

<u>Méthode 2</u> : Démarrer les instances à partir de fichiers de configuration (recommandé en production)

Terminal 1 – Nœud 1 (futur Primary)

```
PS C:\mongodbRep> mongod --config "C:\mongodbRep\mongodb\config\rs1.cfg"
```

Terminal 2 - Nœud 2 (Secondary)

```
PS C:\mongodbRep> mongod --config "C:\mongodbRep\mongodb\config\rs2.cfg"
```

Terminal 3 - Nœud 3 (Secondary)

```
PS C:\mongodbRep> mongod --config "C:\mongodbRep\mongodb\config\rs3.cfg"
```

> *Exécuter les commandes dans le terminal dans les deux méthodes nécessite le maintien simultané de trois terminaux ouverts pendant toute la durée des tests. Toute fermeture accidentelle d'un terminal entraîne l'arrêt du nœud correspondant et peut fausser les résultats obtenus et consomme les ressources.*
> ***Plutôt que de garder 3 fenêtres ouvertes, on enregistre chaque nœud comme service Windows qui démarre automatiquement***

➢ **Installation de services pour chaque nœud**

```
PS C:\mongodbRep> mongod --config "C:\mongodbRep\mongodb\config\rs1.cfg" --serviceName "MongoDB_RS1" --serviceDisplayName "MongoDB ReplicaSet Node 1" --inst
all
{"t":{"$date":"2026-05-01T15:22:43.532+00:00"},"s":"I",  "c":"-",        "id":8991200, "ctx":"thread1","msg":"Shuffling initializers","attr":{"seed":3464337
64}}
PS C:\mongodbRep> mongod --config "C:\mongodbRep\mongodb\config\rs2.cfg" --serviceName "MongoDB_RS2" --serviceDisplayName "MongoDB ReplicaSet Node 2" --inst
all
{"t":{"$date":"2026-05-01T15:22:50.081+00:00"},"s":"I",  "c":"-",        "id":8991200, "ctx":"thread1","msg":"Shuffling initializers","attr":{"seed":2534247
410}}
PS C:\mongodbRep> mongod --config "C:\mongodbRep\mongodb\config\rs3.cfg" --serviceName "MongoDB_RS3" --serviceDisplayName "MongoDB ReplicaSet Node 3" --inst
all
{"t":{"$date":"2026-05-01T15:22:55.351+00:00"},"s":"I",  "c":"-",        "id":8991200, "ctx":"thread1","msg":"Shuffling initializers","attr":{"seed":3686366
337}}
PS C:\mongodbRep> mongod --config "C:\mongodbRep\mongodb\config\arbiter.cfg" --serviceName "MongoDB_ARBITER" --serviceDisplayName "MongoDB ReplicaSet Arbite
r" --install
{"t":{"$date":"2026-05-01T15:23:00.013+00:00"},"s":"I",  "c":"-",        "id":8991200, "ctx":"thread1","msg":"Shuffling initializers","attr":{"seed":2480288
746}}
PS C:\mongodbRep> Start-Service MongoDB_RS1, MongoDB_RS2, MongoDB_RS3, MongoDB_ARBITER
PS C:\mongodbRep> Get-Service MongoDB_RS1, MongoDB_RS2, MongoDB_RS3, MongoDB_ARBITER

Status   Name               DisplayName
------   ----               -----------
Running  MongoDB_ARBITER    MongoDB ReplicaSet Arbiter
Running  MongoDB_RS1        MongoDB ReplicaSet Node 1
Running  MongoDB_RS2        MongoDB ReplicaSet Node 2
Running  MongoDB_RS3        MongoDB ReplicaSet Node 3
```

La création de services pour chacun d'eux permet de démarrer un service juste quand on en a besoin et permet aussi d'éviter d'avoir 3 fenêtres de terminal pour chaque nœud.

5. Initialisation du réplicaset

Se connecter au *Noeud 1* via mongosh :

```
PS C:\mongodbRep> mongosh --port 27017
Current Mongosh Log ID: 69f4c5c644229efe4ba901ad
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+1.2.2
Using MongoDB:          8.2.6
Using Mongosh:          1.2.2

For mongosh info see: https://docs.mongodb.com/mongodb-shell/

------
   The server generated these startup warnings when booting:
   2026-05-01T15:23:30.050+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
------

test> db.serverCmdLineOpts()
{
  argv: [
    'C:\\Program Files\\MongoDB\\Server\\8.2\\bin\\mongod.exe',
    '--config',
    'C:\\mongodbRep\\mongodb\\config\\rs1.cfg',
    '--service'
  ],
  parsed: {
    config: 'C:\\mongodbRep\\mongodb\\config\\rs1.cfg',
    net: { bindIp: '127.0.0.1', port: 27017 },
    replication: { replSetName: 'rs0' },
    service: true,
    storage: { dbPath: 'C:\\mongodbRep\\data\\rs1' },
    systemLog: {
      destination: 'file',
      logAppend: true,
      path: 'C:\\mongodbRep\\logs\\mongodb\\rs1.log'
    }
  },
  ok: 1
}
```

- **Lancer la commande d'initialisation avec la configuration des trois membres :**

```
test> rs.initiate({ _id: "rs0", members: [ { _id: 0, host: "localhost:27017", priority: 2 }, { _id: 1, host: "localhost:27018", priority: 1 }, { _id: 2, hos
t: "localhost:27019", priority: 1 }] })
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1777649212, i: 1 }),
    signature: {
      hash: Binary(Buffer.from("0000000000000000000000000000000000000000", "hex"), 0),
      keyId: Long("0")
    }
  },
  operationTime: Timestamp({ t: 1777649212, i: 1 })
}
```

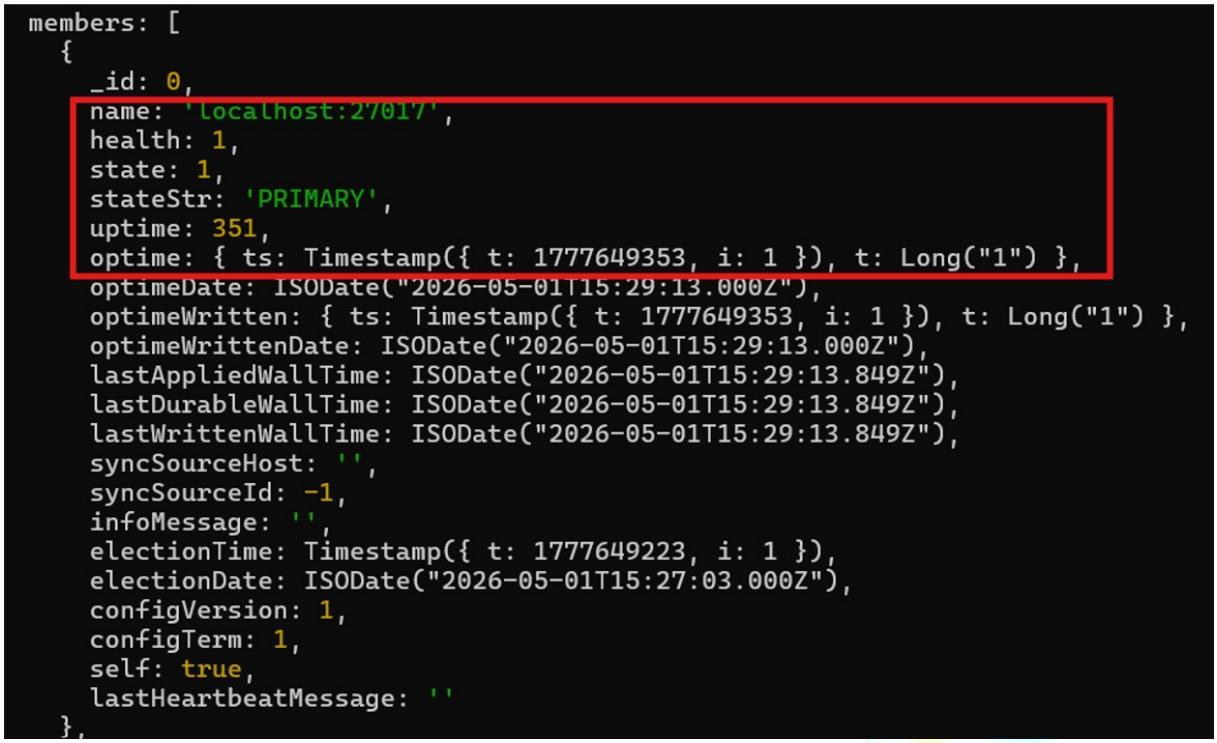
- **Résultat obtenus :**



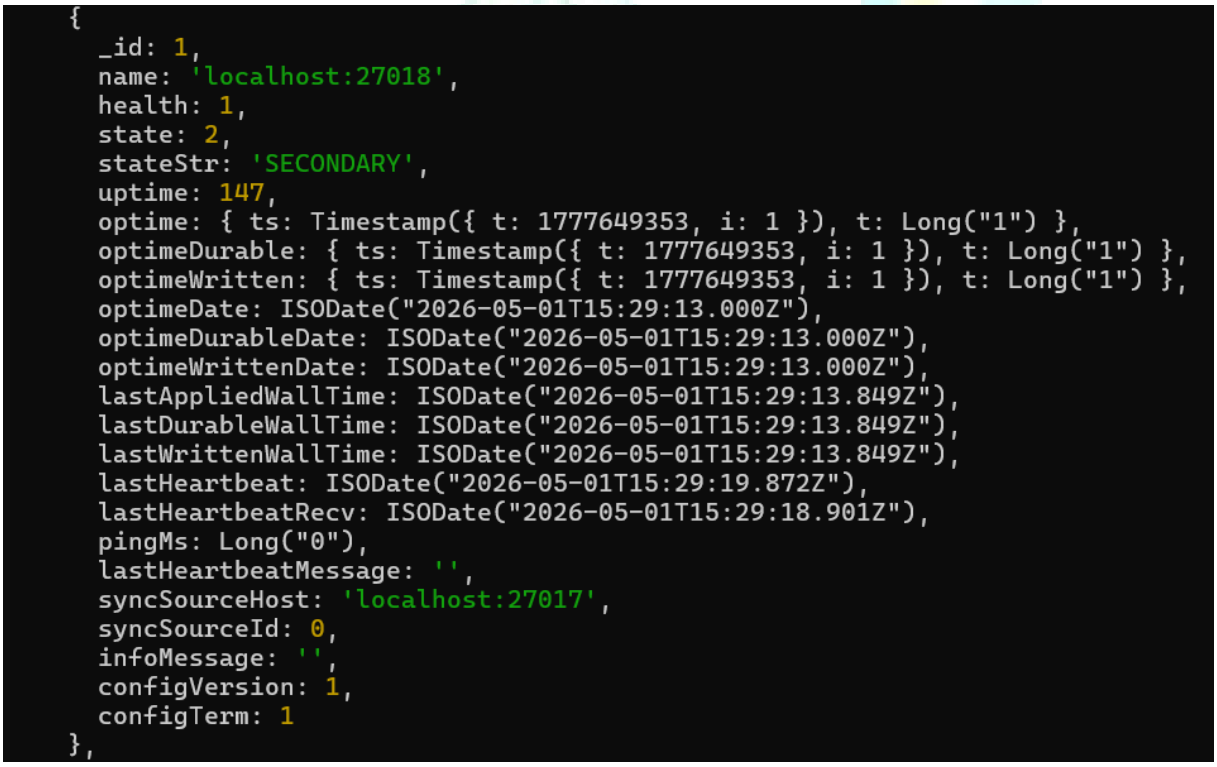
6. Vérification des statuts du ReplicaSet

Vérifier que le ReplicaSet est correctement configuré :

```
rs0 [direct: secondary] test> rs.status()
{
  set: 'rs0',
  date: ISODate("2026-05-01T15:29:20.060Z"),
  myState: 1,
  term: Long("1"),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long("2000"),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1777649353, i: 1 }), t: Long("1") },
    lastCommittedWallTime: ISODate("2026-05-01T15:29:13.849Z"),
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1777649353, i: 1 }), t: Long("1") },
    appliedOpTime: { ts: Timestamp({ t: 1777649353, i: 1 }), t: Long("1") },
    durableOpTime: { ts: Timestamp({ t: 1777649353, i: 1 }), t: Long("1") },
    writtenOpTime: { ts: Timestamp({ t: 1777649353, i: 1 }), t: Long("1") },
    lastAppliedWallTime: ISODate("2026-05-01T15:29:13.849Z"),
    lastDurableWallTime: ISODate("2026-05-01T15:29:13.849Z"),
    lastWrittenWallTime: ISODate("2026-05-01T15:29:13.849Z")
  },
  lastStableRecoveryTimestamp: Timestamp({ t: 1777649323, i: 1 }),
  electionCandidateMetrics: {
    lastElectionReason: 'electionTimeout',
    lastElectionDate: ISODate("2026-05-01T15:27:03.770Z"),
    electionTerm: Long("1"),
    lastCommittedOpTimeAtElection: { ts: Timestamp({ t: 1777649212, i: 1 }), t: Long("-1") },
    lastSeenWrittenOpTimeAtElection: { ts: Timestamp({ t: 1777649212, i: 1 }), t: Long("-1") },
    lastSeenOpTimeAtElection: { ts: Timestamp({ t: 1777649212, i: 1 }), t: Long("-1") },
    numVotesNeeded: 2,
    priorityAtElection: 2,
    electionTimeoutMillis: Long("10000"),
    numCatchUpOps: Long("0"),
    newTermStartDate: ISODate("2026-05-01T15:27:03.817Z"),
    wMajorityWriteAvailabilityDate: ISODate("2026-05-01T15:27:04.309Z")
  },
```



```
members: [
  {
    _id: 0,
    name: 'localhost:27017',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 351,
    optime: { ts: Timestamp({ t: 1777649353, i: 1 }), t: Long("1") },
    optimeDate: ISODate("2026-05-01T15:29:13.000Z"),
    optimeWritten: { ts: Timestamp({ t: 1777649353, i: 1 }), t: Long("1") },
    optimeWrittenDate: ISODate("2026-05-01T15:29:13.000Z"),
    lastAppliedWallTime: ISODate("2026-05-01T15:29:13.849Z"),
    lastDurableWallTime: ISODate("2026-05-01T15:29:13.849Z"),
    lastWrittenWallTime: ISODate("2026-05-01T15:29:13.849Z"),
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    electionTime: Timestamp({ t: 1777649223, i: 1 }),
    electionDate: ISODate("2026-05-01T15:27:03.000Z"),
    configVersion: 1,
    configTerm: 1,
    self: true,
    lastHeartbeatMessage: ''
  },
```



```
  {
    _id: 1,
    name: 'localhost:27018',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 147,
    optime: { ts: Timestamp({ t: 1777649353, i: 1 }), t: Long("1") },
    optimeDurable: { ts: Timestamp({ t: 1777649353, i: 1 }), t: Long("1") },
    optimeWritten: { ts: Timestamp({ t: 1777649353, i: 1 }), t: Long("1") },
    optimeDate: ISODate("2026-05-01T15:29:13.000Z"),
    optimeDurableDate: ISODate("2026-05-01T15:29:13.000Z"),
    optimeWrittenDate: ISODate("2026-05-01T15:29:13.000Z"),
    lastAppliedWallTime: ISODate("2026-05-01T15:29:13.849Z"),
    lastDurableWallTime: ISODate("2026-05-01T15:29:13.849Z"),
    lastWrittenWallTime: ISODate("2026-05-01T15:29:13.849Z"),
    lastHeartbeat: ISODate("2026-05-01T15:29:19.872Z"),
    lastHeartbeatRecv: ISODate("2026-05-01T15:29:18.901Z"),
    pingMs: Long("0"),
    lastHeartbeatMessage: '',
    syncSourceHost: 'localhost:27017',
    syncSourceId: 0,
    infoMessage: '',
    configVersion: 1,
    configTerm: 1
  },
```

La commande **rs.status()** retourne des informations détaillées sur chaque membre, notamment :

- **name** : l'adresse host:port du noeud
- **stateStr** : PRIMARY, SECONDARY, ou ARBITER
- **health** : 1 si le noeud est en bonne santé, 0 sinon
- **optime** : horodatage de la dernière opération répliquée

III. Tests et Résultats

1. Test 1 — Réplication des données

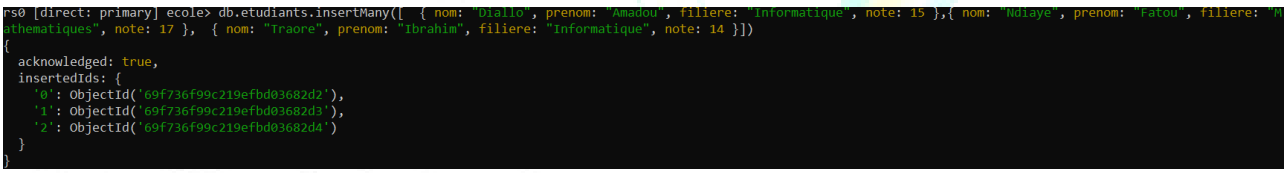
Description du test

Ce test vérifie que les données insérées sur le Primary sont automatiquement répliquées sur les nœuds secondaires.

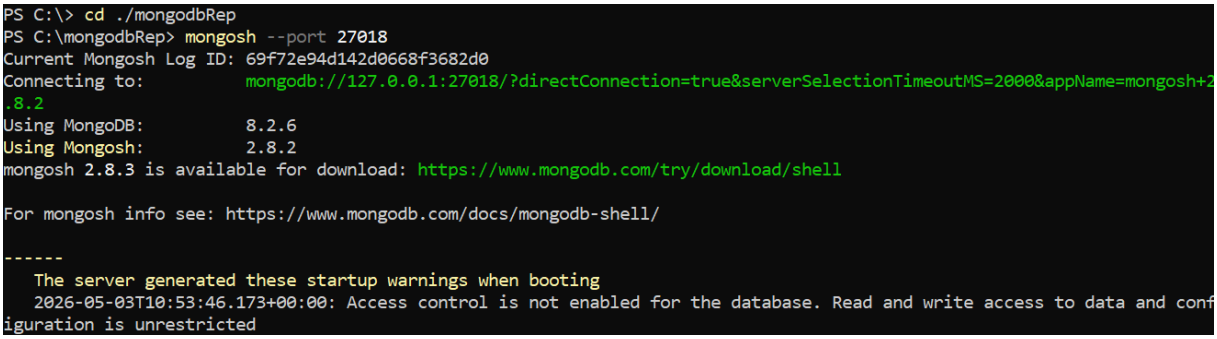
Procédure

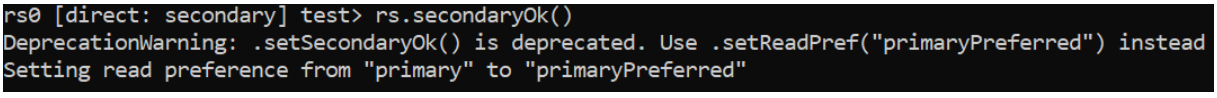
Etape 1 : Se connecter au Primary (port 27017) et insérer des données :

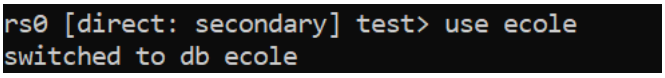




Etape 2 : Se connecter au Secondary (port 27018) et vérifier la réplication :







```
rs0 [direct: secondary] ecole> db.etudiants.find().pretty()
[
  {
    _id: ObjectId('69f736f99c219efbd03682d3'),
    nom: 'Ndiaye',
    prenom: 'Fatou',
    filiere: 'Mathematiques',
    note: 17
  },
  {
    _id: ObjectId('69f736f99c219efbd03682d4'),
    nom: 'Traore',
    prenom: 'Ibrahim',
    filiere: 'Informatique',
    note: 14
  },
  {
    _id: ObjectId('69f736f99c219efbd03682d2'),
    nom: 'Diallo',
    prenom: 'Amadou',
    filiere: 'Informatique',
    note: 15
  }
]
```

Résultats et Observations

Résultat obtenus :

Les trois documents insérés sur le Primary apparaissent immédiatement sur le Secondary après quelques millisecondes. La réplication est transparente et automatique.

Observations :

- La réplication s'effectue en temps quasi réel (latence très faible en local, non chronométrée dans ce test).
- La commande **rs.secondaryOk()** est obligatoire pour autoriser les lectures sur un Secondary.
- Sans cette commande, MongoDB retourne une erreur : 'not primary and secondaryOk=false'.

2. Test 2 — Failover (Panne du Primary)

Description du test

Ce test simule une panne du nœud Primary et vérifie que le ReplicaSet procède automatiquement à l'élection d'un nouveau Primary pour maintenir la disponibilité du service.

Procédure

Etape 1 : Vérifier l'état initial du ReplicaSet :

```
rs0 [direct: primary] ecole> db.etudiants.find().pretty()

rs0 [direct: primary] ecole> rs.status().myState
1
rs0 [direct: primary] ecole> db.getName()
ecole
rs0 [direct: primary] ecole> db.etudiants.insert({nom: "Test"})
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('69f7364b9c219efbd03682d1') }
}
rs0 [direct: primary] ecole> db.etudiants.find()
[ { _id: ObjectId('69f7364b9c219efbd03682d1'), nom: 'Test' } ]
```

Etape 2 : Simuler la panne en arrêtant le processus Primary :

```
PS C:\mongodbRep> mongosh --port 27017
Current Mongosh Log ID: 69f729b09c219efbd03682d0
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.2
Using MongoDB:          8.2.6
Using Mongosh:          2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-05-03T10:53:17.883+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
------
```

```
rs0 [direct: primary] ecole> db.adminCommand({ shutdown: 1})
Uncaught:
MongoNetworkError: read ECONNRESET
Caused by:
Error: read ECONNRESET
ecole>
```

Etape 3 : Observer l'élection sur les nœud restants :

Ça veut dire que le nœud 27018 est devenu automatiquement le nouveau Primary après la panne ! **L'élection a bien fonctionné !**

```
PS C:\mongodbRep> mongosh --port 27018
Current Mongosh Log ID: 69f7392a0f3061cb843682d0
Connecting to:          mongodb://127.0.0.1:27018/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.2
Using MongoDB:          8.2.6
Using Mongosh:          2.8.2
mongosh 2.8.3 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-05-03T10:53:46.173+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
------

rs0 [direct: primary] test> rs.status()
Uncaught:
BSONError: Invalid UTF-8 string in BSON document
Caused by:
TypeError: The encoded data was not valid for encoding utf-8
rs0 [direct: primary] test> rs.status().members.forEach(m => print(m.name, m.stateStr))
Uncaught:
BSONError: Invalid UTF-8 string in BSON document
Caused by:
TypeError: The encoded data was not valid for encoding utf-8
```

```
ecole> rs.status()
Uncaught:
MongoServerSelectionError: connect ECONNREFUSED 127.0.0.1:27017
Caused by:
MongoNetworkError: connect ECONNREFUSED 127.0.0.1:27017
Caused by:
Error: connect ECONNREFUSED 127.0.0.1:27017
ecole>
```

Résultats et Observations

Résultat obtenus

Après le délai d'élection (electionTimeoutMillis, 10 secondes par défaut), l'un des Secondaries (le nœud 27018, de priorité 1) est élu nouveau Primary. L'autre Secondary reste Secondary. Le service est rétabli automatiquement.

Observations :

- Le délai d'élection est déterminé par le paramètre **electionTimeoutMillis** (10 secondes par défaut).
- Pendant la période d'élection, les écritures sont temporairement indisponibles.
- Les lectures configurées avec **readPreference:** secondary restent disponibles pendant ce délai.

- Après élection, le nœud 27018 affiche **stateStr**: PRIMARY dans **rs.status().**

3. Test 3 — Réintégration d'un nœud

Description du test

Ce test vérifie que le nœud qui était Primary, après avoir été relancé, rejoint automatiquement le ReplicaSet comme Secondary et se resynchronise avec les données manquantes.

Procédure

Etape 1 : Insérer de nouvelles données pendant que le *nœud 1* est éteint :

```
PS C:\> cd ./mongodbRep
PS C:\mongodbRep> mongosh --port 27018
Current Mongosh Log ID: 69f72e94d142d0668f3682d0
Connecting to:          mongodb://127.0.0.1:27018/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2
.8.2
Using MongoDB:          8.2.6
Using Mongosh:          2.8.2
mongosh 2.8.3 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-05-03T10:53:46.173+00:00: Access control is not enabled for the database. Read and write access to data and conf
iguration is unrestricted
```

```
rs0 [direct: primary] test> use ecole
switched to db ecole
rs0 [direct: primary] ecole> db.etudiants.insertOne({ nom: "Ba", prenom: "Mariama", filiere: "Physique", note: 18})
{
  acknowledged: true,
  insertedId: ObjectId('69f73be60f3061cb843682d1')
}
rs0 [direct: primary] ecole>
```

Etape 2 : Relancer le *nœud 1* :

```
Start-Service MongoDB_RS1   # nœud 1 installé comme service
(équivalent : mongod --config "C:\mongodbRep\mongodb\config\rs1.cfg")
```

Etape 3 : Vérifier la réintégration et la synchronisation :

```
PS C:\> cd ./mongodbRep
PS C:\mongodbRep> mongosh --port 27017
Current Mongosh Log ID: 69f73ca03baae7837e3682d0
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.2
Using MongoDB:          8.2.6
Using Mongosh:          2.8.2
mongosh 2.8.3 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-05-03T12:15:31.557+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
------
```

```
rs0 [direct: primary] test> rs.secondaryOk()
DeprecationWarning: .setSecondaryOk() is deprecated. Use .setReadPref("primaryPreferred") instead
Setting read preference from "primary" to "primaryPreferred"

rs0 [direct: primary] test> use ecole
switched to db ecole
rs0 [direct: primary] ecole> db.etudiants.find().pretty()
[
  {
    _id: ObjectId('69f736f99c219efbd03682d2'),
    nom: 'Diallo',
    prenom: 'Amadou',
    filiere: 'Informatique',
    note: 15
  },
  {
    _id: ObjectId('69f736f99c219efbd03682d3'),
    nom: 'Ndiaye',
    prenom: 'Fatou',
    filiere: 'Mathematiques',
    note: 17
  },
  {
    _id: ObjectId('69f736f99c219efbd03682d4'),
    nom: 'Traore',
    prenom: 'Ibrahim',
    filiere: 'Informatique',
    note: 14
  },
  {
    _id: ObjectId('69f73be60f3061cb843682d1'),
    nom: 'Ba',
    prenom: 'Mariama',
    filiere: 'Physique',
    note: 18
  }
]
```

Résultats et Observations

Résultat obtenus

Le nœud relancé rejoint automatiquement le ReplicaSet, se resynchronise via l'oplog et récupère le document inséré pendant son absence. Toutes les données sont cohérentes sur les trois nœuds.

Observations :

- La resynchronisation est automatique via la lecture de l'oplog du nouveau Primary.
- Si le nœud a été éteint trop longtemps (plus de la taille de l'oplog), une resynchronisation initiale complète (initial sync) est déclenchée.

- Le nœud 27017 a la priorité la plus élevée (2, contre 1 pour 27018 et 27019) : une fois rattrapé, il peut reprendre le rôle de Primary. Les captures du test 4 le montrent à l'état PRIMARY (port 27017).

4. Test 4 — Lecture sur les nœuds Secondaires

Description du test

Ce test vérifie la configuration de la lecture distribuée sur les nœuds Secondaries pour démontrer la répartition de charge.

Procédure

```
PS C:\> cd ./mongodbRep
PS C:\mongodbRep> mongosh --port 27018
Current Mongosh Log ID: 69f72e94d142d0668f3682d0
Connecting to:          mongodb://127.0.0.1:27018/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+
.8.2
Using MongoDB:          8.2.6
Using Mongosh:          2.8.2
mongosh 2.8.3 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-05-03T10:53:46.173+00:00: Access control is not enabled for the database. Read and write access to data and con
iguration is unrestricted
```

```
rs0 [direct: primary] ecole> db.getMongo().setReadPref("secondary")

rs0 [direct: primary] ecole> use ecole
already on db ecole
rs0 [direct: primary] ecole> db.etudiants.find({filiere: "Informatique"}).pretty()
[
  {
    _id: ObjectId('69f736f99c219efbd03682d2'),
    nom: 'Diallo',
    prenom: 'Amadou',
    filiere: 'Informatique',
    note: 15
  },
  {
    _id: ObjectId('69f736f99c219efbd03682d4'),
    nom: 'Traore',
    prenom: 'Ibrahim',
    filiere: 'Informatique',
    note: 14
  }
]
```

Vérifier depuis quelle instance la lecture est effectuée :

```
rs0 [direct: primary] ecole> db.etudiants.find().explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'ecole.etudiants',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: '4E45B701',
    planCacheShapeHash: '4E45B701',
    planCacheKey: '36FF8BFB',
    optimizationTimeMillis: 6,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: { isCached: false, stage: 'COLLSCAN', direction: 'forward' },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 11,
    totalKeysExamined: 0,
    totalDocsExamined: 4,
    executionStages: {
      isCached: false,
      stage: 'COLLSCAN',
      nReturned: 4,
      executionTimeMillisEstimate: 0,
      works: 5,
      advanced: 4,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      direction: 'forward',
      docsExamined: 4
    }
```

```
queryShapeHash: '84AC7816313EE9521C5DB88D30395251D5DF7434AF7D55A59329C85313DB3FBC',
command: { find: 'etudiants', filter: {}, '$db': 'ecole' },
serverInfo: {
  host: '<masqué>'
  port: 27017,
  version: '8.2.6',
  gitVersion: '5d25c835745d06f712320b6cdae9d50b7b43663e'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFrameworkControl: 'trySbeRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
ok: 1,
'$clusterTime': {
  clusterTime: Timestamp({ t: 1777814926, i: 1 }),
  signature: {
    hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
    keyId: Long('0')
  }
},
operationTime: Timestamp({ t: 1777814926, i: 1 })
```

Résultats et Observations

Résultat obtenus

La préférence de lecture secondary a été configurée (setReadPref) et la requête retourne les documents attendus. Limite : la capture explain() indique serverInfo.port 27017, c'est-à-dire le nœud Primary, atteint par une connexion directe ; elle ne prouve donc pas la redirection vers un Secondary. Pour la démontrer, il faut se connecter avec la chaîne du ReplicaSet (mongodb://localhost:27017,localhost:27018,localhost:27019/?replicaSet=rs0&readPreference=secondary) et vérifier le port dans serverInfo. Lorsque la lecture est effectivement dirigée vers un Secondary, la charge du Primary diminue.

Observations :

- La lecture sur Secondary peut retourner des données légèrement en retard (**eventual consistency**) si la réplication n'est pas encore complétée.

- Pour les applications nécessitant une cohérence forte, il faut utiliser readPreference: primary.
- La distribution des lectures est particulièrement utile pour les tableaux de bord, les rapports et les requêtes analytiques.

IV. LIMITES ET PERSPECTIVES

1. Limites du ReplicaSet seul

Bien que le ReplicaSet offre une excellente haute disponibilité, il présente des limitations importantes lorsqu'il est utilisé seul :

- Limites en matière d'écritures
  - Goulot d'étranglement unique : toutes les écritures transitent par un seul nœud (le Primary). En cas de charge d'écriture élevée, le Primary devient le point de contention.
  - Pas de scalabilité horizontale en écriture : ajouter des Secondaries n'améliore pas les performances d'écriture.

- Limites en matière de volume de données
  - Un seul jeu de données : tous les nœuds du ReplicaSet stockent l'intégralité des données. Pour des volumes de plusieurs To, cela devient problématique en termes de stockage et de RAM.
  - Pas de partitionnement natif des données entre les nœuds.

- Autres limites
  - Cohérence éventuelle : les lectures sur les Secondaries peuvent retourner des données légèrement antérieures aux dernières écritures.
  - Durée d'élection : le basculement (de l'ordre de la dizaine de secondes, selon electionTimeoutMillis) peut être trop long pour certaines applications critiques.
  - Arbiter déconseille en production cloud (interdit sur MongoDB Atlas).

Limite	Impact	Solution
Ecriture sur Primary unique	Goulot d'étranglement	**Sharded Cluster**
Volume de données	Stockage identique partout	**Sharding**
Cohérence éventuelle	Lecture stale possible	writeConcern majority
Durée d'élection	Une dizaine de secondes d'indisponibilité en écriture	Optimiser electionTimeoutMillis

2. Combinaison possible avec le Sharding

Pour répondre aux limitations du ReplicaSet seul, MongoDB propose le Sharded Cluster : une architecture qui combine le Sharding (distribution des données) et les ReplicaSets (haute disponibilité par shard).

Architecture d'un Sharded Cluster MongoDB

- Shards : chaque shard est lui-même un ReplicaSet (haute disponibilité garantie par shard)
- Config Servers : stockent les métadonnées de distribution (eux aussi déployés en ReplicaSet)
- mongos : routeur qui redirige les requêtes vers les bons shards

Un Sharded Cluster avec 3 shards et 3 nœuds par shard = 9 nœud de données minimum. La complexité opérationnelle augmente significativement.

Quand passer au Sharded Cluster ?

- Volume de données supérieur à la capacite d'un seul serveur
- Charge d'écriture ne pouvant être absorbée par un seul Primary
- Besoin de distribution goégraphique des données
- Latence de lecture a optimiser pour des utilisateurs distribues mondialement

Pour la majorité des applications, le ReplicaSet est suffisant et recommande comme point de départ. La migration vers un Sharded Cluster se fait lorsque les limites sont effectivement atteintes.

CONCLUSION

Ce projet nous a permis d'étudier et de mettre en œuvre le mécanisme de ReplicaSet de MongoDB, l'une des fonctionnalités les plus importantes pour garantir la haute disponibilité et la tolérance aux pannes dans les systèmes bases sur MongoDB.

A travers l'étude théorique, nous avons compris les concepts fondamentaux : le rôle complémentaire du Primary, des Secondaries et de l'Arbiter ; le mécanisme de l'Oplog comme journal de réplication idempotent ; le processus d'élection automatique base sur le quorum ; et la flexibilité offerte par les Read/Write Concerns pour adapter le compromis cohérence/performance selon les besoins.

La phase de déploiement et de tests a confirmé la robustesse du système : la réplication est quasi-instantanée en environnement local, le failover s'opère automatiquement, après le délai d'élection (10 secondes par défaut), sans intervention manuelle, et la réintégration d'un nœud se fait de manière transparente via le mécanisme de rattrapage Oplog.

Nous avons également identifié les limites du ReplicaSet seul, notamment l'absence de scalabilité horizontale en écriture et la contrainte de stocker l'intégralité des données sur chaque nœud. La solution naturelle à ces limitations le Sharded Cluster constitue la prochaine étape logique pour les architectures MongoDB à grande échelle.

En conclusion, la maitrise du ReplicaSet MongoDB est une compétence essentielle pour tout architecte ou développeur travaillant avec MongoDB en environnement de production. Les concepts étudiés ici réplication, élection, Oplog, concerns se retrouvent dans l'ensemble de l'écosystème MongoDB distribué.

FIN